

UCS503P – Software Engineering

Vigilante

A Personalized Email Prioritization System

An Autonomous, Cloud-Deployed Prototype

Submitted by:

1024030679 Aryan Sharma

1024030687 Maithli Arora

1024030676 Tushar Sharma

Group : 3C5A , BE Third Year, CSE

Submitted to:

Dr. Raghav B. Venkataramaiyer

Prototype Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	2
1.1	Background and Context	2
1.1.1	Problem Statement	2
1.2	Project Objectives	2
2	Methodology and Design	4
2.1	System Architecture	4
2.1.1	High-Level Design	4
2.1.2	Technical Stack	4
2.2	Implementation Details	5
2.2.1	Module 1: The Baseline Classifier	5
2.2.2	Module 2: The Operational Pipeline	5
3	Results and Evaluation	7
3.1	Testing Methodology	7
3.2	Performance Metrics	7
3.3	Analysis	8
4	Conclusion	9
4.1	Summary of Work	9
4.2	Key Findings	9
4.3	Future Work and Recommendations	9
4.4	Final Remarks	10

Introduction

1.1 Background and Context

Electronic mail remains one of the most widely used forms of digital communication, yet the modern inbox has become difficult to manage. An average user receives a large volume of mail each day, the majority of which is promotional content, newsletters, and automated notifications. The small number of genuinely important messages — a security alert, a financial transaction, a personal reply, or a deadline — are frequently buried within this noise and are easily missed.

Existing email clients offer basic filtering (for example, category tabs and rule-based filters), but these mechanisms are generic and static. They apply the same logic to every user and cannot learn what a specific individual considers important. As a result, users continue to spend time manually triaging their inboxes.

This project, *Vigilante*, addresses this problem by building a system that connects to a user's inbox, automatically judges the importance of each incoming email using a machine-learning model, surfaces the messages that matter, and removes clearly low-value mail — all without requiring continuous user intervention.

1.1.1 Problem Statement

Users are unable to reliably identify important emails among the large volume of low-value mail they receive, and existing filters cannot personalise this judgement. The specific challenges are: (i) important mail is lost among high-volume promotional and automated content; (ii) one-size-fits-all filters cannot capture an individual's notion of importance; (iii) static, rule-based systems fail on modern mail, where a critical message and a marketing message can share the same superficial characteristics (for example, both arriving from a **no-reply** address); and (iv) no existing mechanism adapts over time as a user's priorities change. The problem is therefore to design a system that classifies email importance accurately, acts on that classification safely, and operates continuously and autonomously.

1.2 Project Objectives

1. **Build an accurate importance classifier.** Train a machine-learning model that classifies emails as important or unimportant, using a dataset of real emails labelled and verified by the team, targeting a high recall on the important class so that genuinely important mail is rarely missed.
2. **Establish a secure, persistent inbox connection.** Implement a mechanism by which a user authenticates once and the system maintains access to classify incoming mail continuously, without requiring the user to remain signed in.
3. **Deliver an autonomous, cloud-deployed prototype.** Build an end-to-end system that fetches new mail, classifies it, surfaces important messages, and automatically

archives clearly low-value mail in a safe and reversible manner — deployed to run continuously in the cloud, independent of any local machine.

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

Vigilante is designed as a two-layer intelligence system. The first layer is a **baseline classifier** that provides a generic, cold-start judgement of importance for any new user. The second, future layer is a **personalisation component** (a contextual bandit) that adapts the baseline’s judgement to each individual over time. The present prototype implements and deploys the baseline classifier and the full operational pipeline around it; the personalisation layer is identified as future work.

Operationally, the system is composed of three cooperating cloud services:

- A **scoring service** that hosts the trained model and returns an importance score for any email.
- A **backend and background processor** that manages the secure Gmail connection, fetches new mail, invokes the scoring service, and acts on the results.
- A **scheduler** that triggers the background processor automatically at fixed intervals.

The design deliberately keeps the model’s state in a database rather than in any running process, so that processing can be performed by short-lived, stateless functions — a property that makes the system naturally suited to serverless deployment.

2.1.2 Technical Stack

- **Machine learning:** Python, scikit-learn (logistic regression, TF-IDF vectorisation), pandas.
- **Scoring service:** Python with the FastAPI framework, deployed on Render.
- **Backend:** Node.js with the Express framework.
- **Database:** MongoDB (MongoDB Atlas, cloud-hosted).
- **Email access:** Gmail API with OAuth 2.0 (`gmail.modify` scope).
- **Frontend:** React (Vite), for the prioritised-inbox interface.
- **Cloud deployment:** AWS Lambda (background processor) with Amazon EventBridge (scheduler); Render (scoring service).

2.2 Implementation Details

2.2.1 Module 1: The Baseline Classifier

The core of the system is a machine-learning model that judges email importance. Its development proceeded in three stages: data labelling, human verification, and training.

Data and labelling. A dataset of 6,000 real emails was assembled from two inboxes (personal and college accounts), each email represented by its sender, subject, a short body preview (snippet), and Gmail’s own category labels. Because manually rating 6,000 emails from scratch is impractical and prone to fatigue-induced inconsistency, a rule-based scoring algorithm was designed to produce a consistent first-pass importance score for every email. This algorithm uses a priority cascade: authentication and security messages are scored highest; a promotional/bulk-sender guard suppresses marketing mail; financial, order, personal-reply, and action-required messages are scored as important; and marketing, newsletters, recommendation spam, and social notifications are scored as unimportant. Genuinely ambiguous categories (such as educational and opportunity emails) were placed at the decision boundary and flagged for review.

Human verification. The first-pass labels were exported to a spreadsheet and manually reviewed by the team. Of the 6,000 emails, 1,198 scores (approximately 20%) were corrected, and 854 labels were flipped across the important/unimportant boundary. This human-in-the-loop step is what elevates the dataset from algorithmically-labelled to genuinely verified, and it is the most significant factor in the quality of the resulting model. The final verified distribution was 2,602 important and 3,398 unimportant emails.

Feature engineering and training. A deliberately *realistic* feature set was used, excluding signals that reflect post-arrival user engagement (such as whether an email was starred or already marked important), because those signals would not be available at the moment a new email arrives and would inflate accuracy misleadingly. The features used were: TF-IDF vectors of the subject line (500 features) and of the snippet (500 features), combined with twelve engineered features (automated-sender flag, reply marker, Gmail category flags, text lengths, presence of digits, promotional-word count, and account type). A logistic-regression classifier with balanced class weighting was trained on an 80/20 stratified train-test split.

2.2.2 Module 2: The Operational Pipeline

Around the model, a complete operational pipeline was implemented.

Secure connection. Users connect their Gmail through a read-and-modify OAuth 2.0 flow; the application never receives the account password. During authorisation the system requests offline access and securely stores the resulting long-lived refresh token, which allows the backend to regenerate access on its own indefinitely — so the user authenticates only once.

Background processing with checkpoints. A background processor periodically checks each connected account for new mail. Rather than re-reading the whole inbox, it uses the Gmail History API together with a stored per-user checkpoint (the identifier of the last change processed) to fetch only what is new. After processing, the checkpoint is advanced. This design guarantees that no email is missed or processed twice, and that the system can catch up on any mail that arrived while it was offline. Each new email is sent to the scoring service, and messages scoring below a conservative threshold are archived.

Safe archiving. Archiving is performed by removing a message from the inbox and applying a dedicated `Vigilante/Archived` label. No message is ever deleted; every archiving action is fully reversible, and the applied label makes the system’s actions transparent and visible within the user’s own Gmail.

Interface. A React frontend presents the experience across three screens: a landing page to connect an inbox, a prioritised-inbox view showing each email's importance score and priority, and an archived view where the user can see and restore any message the system archived.

Results and Evaluation

3.1 Testing Methodology

The system was evaluated at two levels: the accuracy of the machine-learning model, and the correctness of the deployed operational pipeline.

- **Model evaluation:** the classifier was measured on a held-out 20% test set (1,200 emails) that it had not been trained on, using accuracy, precision, recall, F1-score, ROC-AUC, and the confusion matrix.
- **Generalisation testing:** the model was additionally stress-tested on freshly written, unseen emails to confirm it produces sensible scores on mail outside the training distribution.
- **Service verification:** the deployed scoring service was verified against a fixed set of known reference scores to confirm that cloud deployment reproduced training-time results exactly.
- **Pipeline testing:** the end-to-end flow (connect, fetch, score, archive, restore) was tested on real, live inboxes, including accounts that were not actively signed in, to confirm autonomous operation.

3.2 Performance Metrics

The baseline classifier achieved the following results on the held-out test set. The model was evaluated in its realistic configuration (excluding post-engagement signals).

Metric	Achieved
Accuracy	0.865
ROC-AUC	0.949
Precision (important class)	0.814
Recall (important class)	0.892
F1-score (important class)	0.851

Table 3.1: Baseline classifier performance on the held-out test set (1,200 emails).

The confusion matrix on the test set was: true negatives 574, false positives 106, false negatives 56, true positives 464. The 56 false negatives (important mail scored as unimportant) represent the most costly error type and were the primary focus of analysis.

The deployed scoring service was verified to reproduce its reference scores on all ten test cases within a tolerance of ± 0.02 , confirming that cloud deployment did not alter model behaviour.

Training data	Recall	Precision	F1
1,200 emails	0.72	0.83	0.77
3,000 emails	0.87	0.92	0.89
6,000 emails (final)	0.90	0.94	0.92

Table 3.2: Improvement of the earlier development model as training data increased.

3.3 Analysis

Meeting the objectives. The primary objective — an accurate importance classifier — was met. A ROC–AUC of 0.949 indicates that the model ranks emails by importance very strongly, which is the property that matters most for a system whose purpose is to prioritise an inbox. The accuracy of 0.865 is an honest, realistic figure precisely because leaky post-engagement signals were deliberately excluded; including them would have produced a higher but misleading number.

Exceeding expectations. The staged training study demonstrated a clear, measurable improvement in recall as data grew (0.72 to 0.90), confirming that the model’s early weakness on rare important categories was a data-quantity problem rather than a fundamental limitation. The system was also successfully deployed to run fully autonomously in the cloud — fetching, classifying, and archiving mail for multiple accounts on a fixed schedule with no local machine involved — which exceeds the typical scope of a prototype.

Challenges and resolutions. Several genuine engineering challenges were encountered and resolved. First, an early version of the model was found, by inspecting its learned weights rather than trusting its accuracy, to have learned a signal in the wrong direction; this was traced to an inconsistency between the labelling rule and a feature definition, and corrected. Second, the model was observed to under-score *quietly* important mail (for example, a plain personal reply); this was addressed by lowering the display threshold to match the score distribution, while keeping the archiving threshold conservative so that no important mail could be wrongly archived. Third, deploying a Python model within a Node.js system required designing a separate scoring microservice, since the two runtimes cannot share a model directly. Fourth, a multi-account bug in the archiving logic (a label identifier cached across accounts) was caught during local testing before deployment. Each of these was identified, understood, and resolved, and the reasoning behind each correction is regarded as a substantive part of the work.

Limitations. The dataset reflects the importance judgements of two inboxes; a broader, more diverse dataset would improve generalisation to arbitrary users. The baseline model is linear and therefore cannot capture complex feature interactions; this is an accepted trade-off for an interpretable cold-start model, and is the motivation for the planned personalisation layer. Finally, because the OAuth application operates in testing mode, connected accounts require re-authorisation periodically, a constraint imposed by the platform rather than the system design.

Conclusion

4.1 Summary of Work

This project set out to build a system that classifies email importance, acts on that classification safely, and operates autonomously. All three objectives were met. A baseline importance classifier was trained on 6,000 human-verified emails and achieved strong ranking performance (ROC-AUC 0.949). A secure, persistent Gmail connection was implemented so that users authenticate only once. A complete operational pipeline — fetching, scoring, and reversible archiving — was built around the model, presented through a React interface, and finally deployed to run continuously and autonomously in the cloud on AWS Lambda, triggered on a fixed schedule and independent of any local machine.

4.2 Key Findings

- **Finding 1:** Human verification of algorithmically-labelled data was the single most important determinant of model quality; correcting roughly 20% of labels materially improved the training signal.
- **Finding 2:** For a prioritisation system, ranking quality (ROC-AUC) is a more meaningful measure of success than binary accuracy, and the model ranks importance strongly even where the binary threshold is imperfect.
- **Finding 3:** Inspecting a model's learned parameters, rather than trusting its headline accuracy, was essential to catching a subtle correctness bug — a reminder that a high accuracy score can conceal a broken model.

4.3 Future Work and Recommendations

1. **Personalisation layer.** Implement the contextual-bandit component that adapts the baseline's scores to each user from their interactions — the main planned enhancement.
2. **Broader dataset.** Train on emails from a more diverse set of inboxes to improve generalisation beyond the two accounts used.
3. **Distribution model.** Because a Gmail-modifying application requires restricted-scope verification for public release, provide the system as an open, self-hostable deployment in which each user retains full ownership of their data and credentials.
4. **Model evolution.** Explore non-linear models (for example, gradient boosting) once the personalisation layer and a larger dataset justify moving beyond a linear baseline.

4.4 Final Remarks

The prototype demonstrates a complete, working, cloud-deployed email prioritisation system built around a genuinely-verified machine-learning model. Beyond the working result, the project reinforced the value of careful, honest engineering: verifying labels by hand, reporting realistic rather than inflated metrics, inspecting a model rather than trusting a single number, and designing actions to be safe and reversible. These principles shaped both the system and the team’s understanding of how to build reliable, trustworthy software.